Experiment No: 01

Libraries in Python for Data Analytics

1. Pandas

A versatile library for data manipulation and analysis, offering powerful data structures like DataFrame and Series.

- Data wrangling: filtering, grouping, merging, and reshaping.
- Handling missing data.
- Reading/writing data from various formats (CSV, Excel, SQL, JSON, etc.).

Example:

```
import pandas as pd

df = pd.read_csv("data/people.csv")

# Filter
filtered = df[df["Age"] > 30]

# Grouping
avg_salary = df.groupby("Age")["Salary"].mean()

print("Filtered Data:\n", filtered)
print("\nAverage Salary:\n", avg_salary)
```

Output:

```
computer@umunuynybtybg:~/Aaryan$ python pandas_demo.py
Filtered Data:
        Name  Age  Salary
2  Charlie   35   70000
3    David   40   80000

Average Salary:
 Age
25    50000.0
30    60000.0
35    70000.0
40    80000.0
Name: Salary, dtype: float64
computer@umunuynybtybg:~/Aaryan$
```

2. NumPy

A fundamental library for numerical computing, providing support for arrays, matrices, and numerical operations.

- Efficient handling of large arrays and matrices.
- Mathematical operations on arrays.
- Random number generation.

Example:

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print("Mean:", np.mean(arr))
print("Standard Deviation:", np.std(arr))
print("Random Numbers:", np.random.randint(1, 100, 5))
```

Output:

```
computer@umunuynybtybg:~/Aaryan$ python numpy_demo.py
Mean: 30.0
Standard Deviation: 14.142135623730951
Random Numbers: [50 71 40 17 16]
computer@umunuynybtybg:~/Aaryan$
```

3. Matplotlib

A library for creating static, animated, and interactive visualizations.

- Supports various chart types (line, bar, scatter, etc.).
- Customizable visualizations.

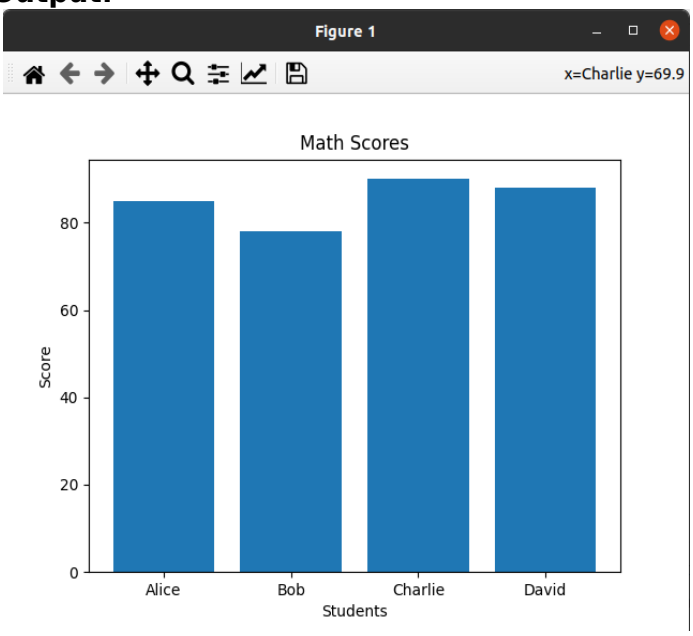
Example:

```
import matplotlib.pyplot as plt
import pandas as pd

df = pd.read_csv("data/scores.csv")

plt.bar(df["Name"], df["Math"])
plt.title("Math Scores")
plt.xlabel("Students")
plt.ylabel("Score")
plt.show()
```

Output:



4. Seaborn

Built on Matplotlib, Seaborn provides a high-level interface for creating beautiful and informative statistical visualizations.

- Automatic handling of aesthetics.
- Built-in functions for visualizing distributions and relationships.

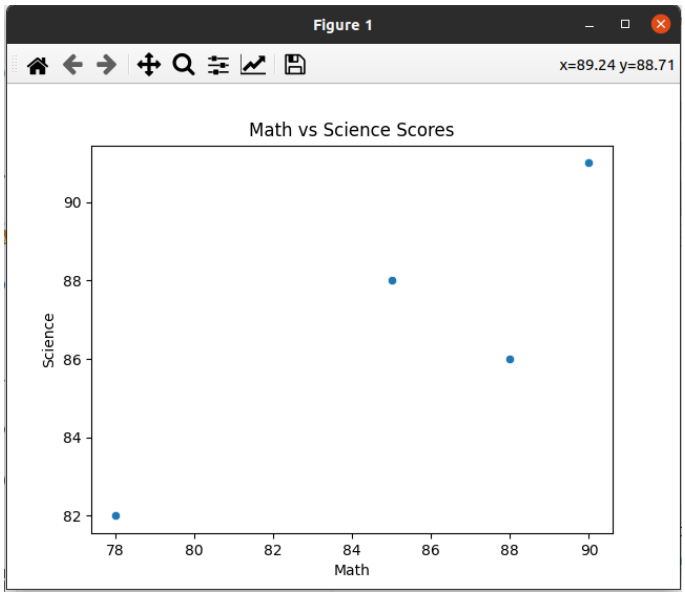
Example:

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data/scores.csv")

sns.scatterplot(x="Math", y="Science", data=df)
plt.title("Math vs Science Scores")
plt.show()
```

Output:



5. Scikit-learn

A library for machine learning and data preprocessing, offering tools for supervised and unsupervised learning.

- Easy implementation of algorithms (e.g., regression, classification, clustering).
- Data preprocessing utilities (e.g., scaling, splitting, feature selection).

Example:

```
from sklearn.linear_model import LinearRegression
import pandas as pd

df = pd.read_csv("data/people.csv")
```

```
X = df[["Age"]]
y = df["Salary"]

model = LinearRegression()
model.fit(X, y)

prediction = model.predict([[45]])
print("Predicted Salary for Age 45:", prediction[0])
```

Output:

```
computer@umunuynybtybg:~/Aaryan$ python sklearn_demo.py
Predicted Salary for Age 45: 90000.0
computer@umunuynybtybg:~/Aaryan$
```

6. SciPy

Extends NumPy with additional modules for scientific and technical computing.

- Modules for optimization, integration, and signal processing.
- Solving linear algebra and differential equations.

Example:

```
from scipy.stats import norm

x = 1
print("Probability Density:", norm.pdf(x, loc=0, scale=1))
```

Output:

```
computer@umunuynybtybg:~/Aaryan$ python scipy_demo.py
Probability Density: 0.24197072451914337
computer@umunuynybtybg:~/Aaryan$
```

7. Plotly

An interactive visualization library that supports both 2D and 3D plotting.

- Interactive and dynamic visualizations.
- Dashboards for data analysis.

Example:

```
import plotly.express as px
import pandas as pd

df = pd.read_csv("data/scores.csv")

fig = px.bar(df, x="Name", y="Science", title="Science Scores")
```

```
fig.show()
```

Output:

8. Statsmodels

A library for statistical modeling and hypothesis testing.

- Regression models (linear, logistic, etc.).
- Time series analysis.

Example:

```
import statsmodels.api as sm
import pandas as pd

df = pd.read_csv("data/people.csv")

X = sm.add_constant(df["Age"])
y = df["Salary"]

model = sm.OLS(y, X).fit()
print(model.summary())
```

Output:

```
computer@umunuynybtybg:~/Aaryan$ python statsmodels_demo.py
/home/computer/.local/lib/python3.8/site-packages/statsmodels/stats/stattools.py:74: ValueWarning: omni_norm
test is not valid with less than 8 observations; 4 samples were given.
  warn("omni_normtest is not valid with less than 8 observations; %i "
                            OLS Regression Results
==============================================================================
Dep. Variable:                 Salary   R-squared:                       1.000
Model:                            OLS   Adj. R-squared:                  1.000
Method:                 Least Squares   F-statistic:                 1.453e+30
Date:                Fri, 09 Jan 2026   Prob (F-statistic):           6.88e-31
Time:                        14:26:00   Log-Likelihood:                 94.553
No. Observations:                   4   AIC:                            -185.1
Df Residuals:                       2   BIC:                            -186.3
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const      -1.455e-11   5.47e-11     -0.266      0.815    -2.5e-10    2.21e-10
Age         2000.0000   1.66e-12   1.21e+15      0.000    2000.000    2000.000
==============================================================================
Omnibus:                          nan   Durbin-Watson:                   0.077
Prob(Omnibus):                    nan   Jarque-Bera (JB):                0.963
Skew:                           1.155   Prob(JB):                        0.618
Kurtosis:                       2.333   Cond. No.                         195.
==============================================================================
```

9. TensorFlow and PyTorch

Primarily used for deep learning, these libraries are also used for advanced data analysis and modeling.

- Handling large datasets with GPUs.
- Advanced modeling for machine learning and neural networks.

Example (TensorFlow):

```
import tensorflow as tf

tensor = tf.constant([[1, 2], [3, 4]])
print("Tensor:\n", tensor)
```

Output:

```
computer@umunuynybtybg:~/Aaryan$ python tensorflow_demo.py
2026-01-09 14:27:56.509444: I tensorflow/core/util/port.cc:110] oneDNN custom operations are on. You may
 see slightly different numerical results due to floating-point round-off errors from different computat
ion orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-01-09 14:27:56.536460: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your
 machine, GPU will not be used.
2026-01-09 14:27:56.730918: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your
 machine, GPU will not be used.
2026-01-09 14:27:56.732024: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary
is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with th
e appropriate compiler flags.
2026-01-09 14:27:57.422785: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Coul
d not find TensorRT
Tensor:
 tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
computer@umunuynybtybg:~/Aaryan$
```

10. OpenCV

A library for image and video processing, often used in data analysis related to computer vision.

- Image transformations, object detection, and feature extraction.
- Works with image datasets.

Example:

```
import cv2

image = cv2.imread("example.jpg")

cv2.imshow("Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output: